CIPHER: Contextual Instruction Precedence and Hierarchy Evaluation for Repositories

Anonymous Authors
Affiliation withheld for review

***Abstract*—Draft: results pending human reference labels.**

***Index Terms*—AI coding agents, AGENTS.md, instruction conflicts, repository mining, scope and precedence**

I. Introduction

AI coding agents read project guidance from natural-language files in the repository. The AGENTS.md convention allows one file per package and states that "the closest AGENTS.md to the edited file wins" [1]. Apparent disagreement between such files is therefore not always a defect. Suppose the root file says "use exactly four spaces per indentation level" and a file in `web/` says "use exactly two". For a new line in the root package the two rules may genuinely conflict; for `web/src/button.ts` the nested rule is a documented local override; and if the second rule lived in `api/`, it would not apply to that target at all. The wording is identical in all three situations; only the task's location and the files' placement differ.

A conflict detector can fail in both directions: it can miss incompatible obligations, or it can report a legitimate local exception as a defect and suggest deleting useful guidance. Existing detectors flag contradictions from text alone. In a study of 100 repositories, a model-based smell detector reported 28 conflicting-instruction cases, of which the first author confirmed 16 [2]; the keyword heuristics of Ruleherd [3] compare files without regard to scope. Whether scope or precedence explains such false alarms has not been tested.

CIPHER asks whether making applicability and precedence explicit improves the classification of apparent conflicts. We contribute (1) a task-conditioned, five-label definition of the problem under a declared AGENTS.md loading profile; (2) a pilot of 27 real instruction pairs from eight repositories at pinned commits, each with a context-changing variant and an order control, labeled independently by two authors and adjudicated by a third; and (3) a comparison of a keyword heuristic, a direct model given the full evidence, and a structured pipeline using the same local model. Section VI reports results once the human reference labels are final. The protocol, data, prompts, raw outputs, and analysis code are released (anonymized artifact link withheld).

II. Related Work

Agent context files have been characterized at scale [4], [5], and their effect on task success is contested [6]. Configuration Smells [2] treats conflicting instructions as a smell detected within one file; Context Rot [7] studies staleness against code. WIRE [8] grounds rules in spans and models activation conditions and decision surfaces, but reports no matched compatible-pair or priority-insertion controls. ContextCov [9] extracts scoped constraints and enforces them, and OctoBench [10] measures which source an agent follows under explicit conflicts. Guidefold [11] already resolves guidance with a most-specific-scope lattice and structural replays, while noting that human-labeled conflicts were still missing. A factorial study found no detectable effect of injected cross-file contradictions on adherence [12], and Arbiter catalogs interference in system prompts [13]. Applicability and precedence reasoning are therefore not new. Our contribution is narrower: human reference labels that separate valid overrides and disjoint applicability from unresolved conflict in mined AGENTS.md hierarchies, with controlled variants, and a same-evidence comparison of a direct and a structured model.

III. Task and Instruction Profile

A *case* is one pair of verbatim instruction spans (with file path, line numbers, heading, and enclosing section), one concrete task that edits one target file, one pinned commit, and one loading profile. The label concerns this pair and task, not the overall quality of a repository.

Profile P1 (frozen; an experimental interpretation of [1], not a claim about every tool). Only AGENTS.md files participate. The binding loaded set is every AGENTS.md on the target's ancestor directory chain; files in unrelated branches are not loaded. A loaded instruction is in force unless its own wording limits it. When two loaded instructions are incompatible on the same decision, the more deeply nested file wins, and ancestor instructions about other decisions stay in force. Equal authority (same file or same depth) is not ranked by order of appearance; only an explicit statement can create priority. User prompts, other file formats, imports, symbolic links, multiple working roots, and multi-target tasks are excluded.

Labels and decision sequence. Annotators and methods follow the same order: (1) if meaning or applicability cannot be established, *Insufficient evidence*; (2) if, before priority, either instruction is inactive (not loaded, or excluded by its own condition), *Disjoint applicability*; (3) if the active instructions can be satisfied together, *Compatible* (different decisions, refinements, duplicates, or a preference beside a requirement); (4) otherwise *Valid override* if P1 or an explicit statement picks a winner, else *Unresolved conflict*. Modality is preserved: "prefer X" does not forbid Y, and descriptions or examples are not rules unless worded as requirements.

IV. Corpus and Annotation

Sampling. The discovery frame was the `context_files.csv` table of a public configuration dataset [5] (Zenodo record 20090356): 237 repositories whose rows show a root and at least one nested AGENTS.md. In a seeded random order we retrieved the first 30 through the GitHub API at their default-branch commit on 21 October 2026, storing every AGENTS.md as inert JSON with its SHA-256. Repositories had to carry an open-source license, not be forks or archived, have no symlinked AGENTS.md, and have a root and a nested file with at least five non-empty lines (24 of 30). Nested files under template, example, vendored, or hidden directories were excluded as separate project roots.

Candidate pairs. Retrieval is separate from classification and deliberately challenge-oriented: instruction units (list items and paragraphs outside code fences that contain a directive verb or modal) are paired when they share a specific term (a tool, command, or convention word) or name different values from one family (for example `npm` and `pnpm`); the score favors different values, opposite polarity, conditions, and precedence words. Verbatim duplicates are excluded. Repositories needed at least four candidate pairs, two of them across files (12 qualified); the first two in frame order formed the calibration split and the next eight the evaluation split, so no repository crosses splits. Up to four pairs per repository were selected by score with a quota for ancestor–descendant and unrelated-branch pairs, yielding 27 evaluation originals (16 ancestor–descendant, 5 unrelated-branch, 6 same-file) and 7 calibration originals. Tasks are researcher-authored from fixed templates (for example "make a small fix in `<target>` and complete the checks this change requires before committing"), and targets are real files under the relevant directory. This selected sample cannot estimate how often conflicts occur.

Strata. Each original has a *context-changing variant* that keeps both instructions' wording and changes one documented property: the target path (16 cases, moving the target outside the nested file's directory) or the placement of one instruction (11 cases, relocating it to the root or to a nested file on the target's chain). Each original also has an *order control* that only swaps the presentation order of the two instructions. Variants are constructed material and are reported separately from originals.

Annotation. Two authors labeled every original and variant independently in a standalone tool, with separately randomized orders, opaque IDs, and no access to predictions, strata, or each other's answers; the tool showed the task, target, profile, the AGENTS.md paths with the target's chain marked, both spans with their sections and full files, and keyword-scanned scope statements. After a 20-item calibration round (six authored practice items), the handbook was frozen. The third author adjudicated disagreements. Order controls inherit their parent's final label after a human check that only the order changed.

V. Methods and Setup

B0 applies Ruleherd's contradiction rules (v0.1.3, commit 94559fc6; keyword pairs such as npm/pnpm and tabs/spaces, and never/always polarity across different files) to the two spans of a case; it yields a binary flag and is scored only on conflict detection. **B1** is a direct contextual model: one call receives the evidence packet (task, target, commit, all AGENTS.md paths, both spans with line-numbered sections, and the scope statements), the full profile, the label definitions, and the decision sequence. **B2**, the CIPHER pipeline, uses the same model and packet. One call extracts, for each instruction, its requirement, regulated decision, modality, explicit condition, and whether that condition covers the task, plus any explicit priority statement. Code then computes the loaded set and depth precedence deterministically. A second call, made only when both instructions remain active and binding, judges whether they constrain the same decision incompatibly; fixed rules map the results to a label. Because B2 may use two calls, **B1R** adds an equal-call control: B1 followed by one self-review call over the same evidence.

All model arms used Qwen3.6-35B-A3B [14] (Q4_K_M, Ollama digest a7eb95c5) locally with thinking enabled, temperature 0.6, top-p 0.95, top-k 20, a fixed seed, JSON-schema-constrained output, and at most two retries with a changed seed. Each request contained only the method prompt and the evidence; no case identifier, stratum, or label. The profile, handbook, case manifest, prompts, schemas, retry policy, and analysis plan were frozen by hash before held-out runs, and held-out predictions stayed sealed until the reference labels were final. The primary outcome is five-class macro-F1 on originals (B2 versus B1) over classes present in the reference; abstentions and failures count as incorrect.

VI. Results

Pending. Human reference labels are not final, so no accuracy is reported. Held-out model runs are sealed.

VII. Threats to Validity

The sample is small (27 originals from eight repositories) and challenge-oriented, so class supports are uneven and some labels are rare; interval estimates over eight repository clusters are descriptive. Tasks were authored from templates, and variants are constructed, so neither represents observed agent sessions. The annotators are authors, which risks shared assumptions; independence, blinding, and adjudication reduce but do not remove this. One profile and one quantized open-weight model limit generalization, and public repositories may appear in its training data. Results concern classification under P1, not whether agents follow instructions or produce better code.

VIII. Conclusion

Pending.

Acknowledgment

Pending.

REFERENCES

[1] "AGENTS.md," https://agents.md/, 2026, format documentation, accessed Oct. 21, 2026.

[2] H. V. F. dos Santos, V. Costa, J. E. Montandon, L. L. Silva, and M. T. Valente, "Configuration smells in AGENTS.md files: Common mistakes in configuring coding agents," in *Proc. 26th IEEE Int. Conf. Source Code Analysis and Manipulation (SCAM)*, 2026, arXiv:2606.15828v5. [Online]. Available: https://arxiv.org/abs/2606.15828v5

[3] princejain756, "ruleherd v0.1.3," GitHub, commit 94559fc6, MIT License, 2026, software, accessed Oct. 21, 2026. [Online]. Available: https://github.com/princejain756/ruleherd

[4] W. Chatlatanagulchai, H. Li, Y. Kashiwa, B. Reid, K. Thonglek, P. Leelaprute, A. Rungsawang, B. Manaskasemsak, B. Adams, A. E. Hassan, and H. Iida, "Agent READMEs: An empirical study of context files for agentic coding," 2026, arXiv:2511.12884v2. [Online]. Available: https://arxiv.org/abs/2511.12884v2

[5] M. Galster, S. Mohsenimofidi, L. Böhme, J. L. Lulla, M. A. Abubakar, C. Treude, and S. Baltes, "A dataset of agentic AI coding tool configurations," in *Proc. 3rd ACM Int. Conf. AI-Powered Software (AIware '26)*, 2026, pp. 314–322.

[6] T. Gloaguen, N. Mündler-Sasahara, M. N. Müller, V. Raychev, and M. Vechev, "Evaluating AGENTS.md: Are repository-level context files helpful for coding agents?" 2026, arXiv:2602.11988v3. [Online]. Available: https://arxiv.org/abs/2602.11988v3

[7] C. Treude and S. Baltes, "Context rot in AI-assisted software development: Repurposing documentation consistency for AI configuration artifacts," 2026, arXiv:2606.09090v1. [Online]. Available: https://arxiv.org/abs/2606.09090v1

[8] L. Yan, X. Chen, and X. Zhang, "WIRE: Profiling witnessed within-policy instruction collisions in LLM agents," 2026, arXiv:2605.27784v2. [Online]. Available: https://arxiv.org/abs/2605.27784v2

[9] R. K. Sharma, "ContextCov: Deriving and enforcing executable constraints from agent instruction files," 2026, arXiv:2603.00822v2. [Online]. Available: https://arxiv.org/abs/2603.00822v2

[10] D. Ding, S. Liu, E. Yang, J. Lin, Z. Chen, S. Dou, H. Guo, W. Cheng, P. Zhao, C. Xiao, Q. Zeng, Q. Zhang, X. Huang, Q. Xu, and T. Gui, "OctoBench: Benchmarking scaffold-aware instruction following in repository-grounded agentic coding," in *Proc. 64th Annu. Meeting Assoc. Comput. Linguistics (Vol. 1: Long Papers)*, 2026, pp. 5958–5978.

[11] wiatrM, "Guidefold: docs/RESEARCH.md," GitHub, commit 1c98b24f, 2026, project notes, accessed Oct. 21, 2026. [Online]. Available: https://github.com/wiatrM/guidefold/blob/1c98b24f8b6fc28a3a7253ab6dd9c3cc626f885e/docs/RESEARCH.md

[12] D. McMillan, "Instruction adherence in coding agent configuration files: A factorial study of four file-structure variables," 2026, arXiv:2605.10039v1. [Online]. Available: https://arxiv.org/abs/2605.10039v1

[13] T. Mason, "Arbiter: Detecting interference in LLM agent system prompts," 2026, arXiv:2603.08993v2. [Online]. Available: https://arxiv.org/abs/2603.08993v2

[14] Qwen Team, "Qwen3.6-35B-A3B," Open-weight model, run locally as Ollama tag qwen3.6:35b-a3b (Q4_K_M, digest a7eb95c5), 2026.